

実装の講評と総括

判断と証拠を振り返る

情報システム学科 3年・選択・2単位

今日の到達目標

- 実装の達成範囲を証拠で説明できる
- 他者の設計を問い、改善案を示せる
- AI利用を含めて学習と開発の変化を振り返れる

前回との接続

Lisp製評価器とREPLをPython製ホスト上で実行した。

今日は新機能の追加より、開発したものの講評を行う。

実装の現在地と次に改善する点を確認する。

講評の対象

コードだけでなく、仕様、受入例、実行ログ、説明を扱う。

動いた結果と、その結果になる理由を結び付ける。

ここでの観点は授業用の案であり、正式な点数配分ではない。

講評の進め方

作者は達成した機能と、未対応の機能を説明する。

聞き手は小さな入力で判断を確かめる。

作者は指摘を受け、改善を1つ選ぶ。

ペアで作者と聞き手を交代する。共通講評の後も、各人のデモと質問の時間を残す。

架空の提出例

以後の例は、本教材が作成した架空の実装。

実在の学生のコード・提出物・成績は使用しない。

例題 `examples/15-showcase/` を議論の材料にする。

例A 算術だけの実装

四則演算と入れ子の式が動く。

変数と関数は未対応だが、失敗条件をREADMEに記載した。

成功例と失敗例から、現在の達成範囲を読み取る。

例Aへの問い

`(+ 1 (* 2 3))` の評価順序を説明できるか。

未知の演算子を渡すと、利用者は原因を理解できるか。

次に変数を加えるなら、どの関数へ環境を渡すか。

例B 環境の混同

関数の呼出しごとに、同じ変数表へ引数を書き込む架空の案。

`f` の引数が、外側の変数を上書きしてしまう。

正常な一例だけではこの誤りを見落とす。

例Bの反例

```
(define x 100)
(define f (lambda (x) (+ x 1)))
(f 4)
x
```

本教材の仕様では結果は 5 と 100。

作者は検索する環境と更新する環境を説明する。

例C Lisp製評価器

対象の `if` と関数をLispで評価できる架空の案。

読取と入出力はPythonホストへ委譲している。

対応機能と委譲範囲を図と実行ログで示した。

例Cへの問い

Lisp製評価器を呼ぶコードはどこか。

非選択枝を評価しない証拠はあるか。

ホストへの依存を残した理由と、その影響を説明できるか。

証拠の比較

証拠	分かること
実行ログ	観察した動作
受入例	守る契約
環境図	名前検索の理由
コード	処理の実装箇所

自分の実装の紹介

対応する入力を1つ選び、結果を予測してから実行する。

その結果を作るコードと環境を説明する。

未対応入力も1つ示し、診断か制限を述べる。

聞き手の質問

作者の説明で曖昧な名前や順序を1つ選ぶ。

小さな反例を提案し、予測を聞く。

作者が実行して確かめられる問いにする。

講評の記録

観察した事実と、改善の提案を分けて書く。

「動かない」だけで終わらせず、入力と実際の結果を残す。

作者の意図を確認してから、設計判断を評価する。

設計の振り返り

変更が少ない箇所と、変更が広がった箇所を比べる。

読取、評価、環境、入出力の境界は役立ったか。

1つの変更を例に、自分の判断を説明する。

データ構造との接続

構文木は再帰的なデータ構造として扱った。

環境探索は、階層をたどる探索になった。

前半の木、探索、計算量がどこで役立ったか挙げる。

検証の振り返り

最初に見落とした反例を1つ選ぶ。

どの契約が曖昧だったかを書き直す。

その反例を、今後も使う回帰検証へ残す。

AI利用の振り返り

AIへ任せた作業と、自分が判断した部分を分ける。

採用前に何を確認したか、具体的な入力で示す。

説明を自分で修正した例も振り返る。

改善案の選択

実装の改善を1つ選び、利用者に起きる変化を書く。

変更箇所と、成功を確かめる受入例を示す。

未実装の案は、完成済みの機能と分けて記述する。

確認問い

結果が正しい一例だけで、設計の正しさを主張できるか。

自分の実装を他者へ渡すとき、どの証拠を一緒に渡すか。

成果物の整理

コード、仕様、受入例、実行ログ、振り返りを対応付ける。

教材の成果物案は `exercises/15.md` にある。

正式な提出先・締切・採点運用は授業で確認する。

全体の総括

ソフトウェアの設計では、必要な動作を契約にし、変更を扱う構造を作る。

プログラムを書き、反例で確かめ、説明を更新する経験を積んだ。

次の開発でも、判断を裏付ける証拠を残す。

参考

- 本教材: `examples/15-showcase/`
- 第1～14回の仕様・演習・振り返り
- [The Scheme Programming Language](#)

外部の言語や処理系と比較するときは、機能差を明記する。